

SWIPEX

AI-Powered Swipe-Based Intelligent Job Discovery
& Career Assistance Platform

Project Report

Prepared by

Nidrashri V

BE Computer Science and Engineering

KPR Institute of Engineering and Technology

2026

Table of Contents

No.	Section
1	Abstract
2	Introduction
3	Problem Statement
4	Objectives
5	Proposed System
6	Major Modules
7	Technology Stack
8	System Architecture
9	Database Design
10	Backend and API Design
11	Frontend Design
12	Deployment
13	Testing and Validation
14	Project Data Migration
15	Future Enhancements
16	Conclusion
17	Repository and Live Deployment

1. Abstract

SwipeX is a web-based intelligent job discovery and career assistance platform designed to simplify the process of discovering, matching, saving and applying for jobs. The platform combines a swipe-oriented job discovery experience with resume/ATS-oriented analysis, job matching, application tracking and recruiter-side job management.

The application uses a JavaScript-based frontend, a Python FastAPI backend, PostgreSQL for persistent data, and SQLAlchemy for database interaction. The production frontend is hosted on Vercel, while the FastAPI backend is hosted on Railway and the production PostgreSQL database is hosted on Neon.

The final deployment was validated through production authentication and job-page routing. Existing local project data was migrated to Neon while preserving relationships between users, jobs, applications, notifications, resume-score history and saved jobs.

2. Introduction

Traditional job portals can require users to repeatedly search, filter and inspect large numbers of listings. SwipeX approaches job discovery as a more interactive workflow. A user can discover jobs, evaluate them against their profile or resume, save relevant opportunities and manage applications from the same platform.

The project also provides a recruiter-oriented workflow for managing posted jobs and viewing candidate/application information. The system therefore contains both job-seeker and recruiter functionality backed by a common API and database.

2.1 Project Scope

- User registration and login.
- Job discovery and search/filter interaction.
- Swipe-oriented job interaction and recommendations.
- Saving jobs and viewing saved opportunities.
- Applying for jobs and tracking application status.
- Resume/ATS score history support.
- Recruiter job management and candidate/application views.
- Notifications and profile-related functionality.

3. Problem Statement

Job seekers often face information overload when searching for opportunities. A large number of listings may not be relevant to their skills, experience or career interests. At the same time, recruiters need mechanisms to manage job postings and applications efficiently.

The project therefore addresses the need for a unified platform that connects job discovery, profile/resume information, job matching, saved jobs and application management in a single workflow.

4. Objectives

- Build a responsive web interface for job discovery.
- Provide secure user authentication and role-based workflows.
- Connect the frontend to a FastAPI REST backend.
- Store application data reliably in PostgreSQL.
- Support job matching using available user/profile/resume information.
- Provide resume/ATS analysis and score history functionality.
- Allow users to save jobs and manage applications.
- Provide recruiter functionality for job and candidate management.
- Deploy the frontend and backend as separate production services.
- Migrate existing project data into the production Neon PostgreSQL database.

5. Proposed System

The proposed SwipeX system separates presentation, application logic and data persistence into distinct layers. The browser-based frontend provides the user interface. FastAPI exposes backend endpoints for authentication, jobs, saved jobs, applications and related operations. PostgreSQL stores persistent records.

5.1 User Roles

- **Job Seeker:** discovers jobs, interacts with recommendations, saves jobs, applies, views application information and manages profile/resume information.
- **Recruiter:** manages job postings and works with candidate/application data.
- **Admin/administrative functionality:** represented in the application architecture where applicable, with the production database supporting role information.

5.2 Core Workflow

Register/Login → Dashboard/Profile → Discover Jobs → Match/Swipe → Save or Apply → Track Applications → Receive/Review Notifications.

6. Major Modules

6.1 Authentication

Registration and login functionality with user role information and backend authentication APIs.

6.2 Job Discovery

Job listings can be loaded from the backend and presented through the Jobs interface with search/filter-oriented interaction.

6.3 Swipe and Matching

The platform uses a swipe-oriented interaction model and matching workflow to help users discover relevant jobs.

6.4 Saved Jobs

Users can save opportunities and retrieve their saved jobs from the database.

6.5 Applications

Users can apply for jobs and maintain application records and statuses.

6.6 Resume and ATS Analysis

Resume information and ATS-score history are represented in the application and persisted for users.

6.7 Recruiter Module

Recruiters can manage jobs and work with application/candidate information.

6.8 Notifications

Notifications are stored with references to users and, where applicable, jobs.

6.9 Profile

User profile functionality provides a central place for personal and resume-related information.

7. Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User interface and client-side interactions
Backend	Python, FastAPI	REST APIs and application logic
ORM	SQLAlchemy	Database models and queries
Database	PostgreSQL / Neon	Production persistent storage
Frontend Hosting	Vercel	Production static frontend hosting
Backend Hosting	Railway	Production FastAPI hosting
Version Control	Git / GitHub	Source control and collaboration

The production architecture intentionally separates frontend hosting from backend hosting. This makes the API independently deployable and allows the frontend to consume the Railway-hosted API.

8. System Architecture

8.1 Production Architecture

Browser → Vercel Frontend → Railway FastAPI Backend → Neon PostgreSQL

The frontend communicates with the production backend using the Railway API base URL. The backend reads the `DATABASE_URL` environment variable and connects to the Neon PostgreSQL instance. Secrets such as database credentials are kept in environment configuration rather than being included in source-controlled code.

8.2 Request Flow Example

- User opens the Jobs page.
- The frontend JavaScript requests job data from the FastAPI `/jobs` endpoint.
- FastAPI accesses PostgreSQL through SQLAlchemy.
- The backend returns job data as JSON.
- The frontend renders the returned jobs in the Jobs interface.

9. Database Design

The production Neon database contains six core tables used by the application.

Table	Purpose
users	Stores user identity, role and resume-related information.
jobs	Stores job title, company, location, salary, experience, skills and description.
applications	Associates users with jobs and stores application status and match percentage.
notifications	Stores user notifications and optional job references.
resume_score_history	Stores ATS scores and resume filenames over time.
saved_jobs	Stores user-to-job saved-job relationships.

9.1 Key Relationships

- applications.user_id → users.id
- applications.job_id → jobs.id
- jobs.posted_by → users.id
- notifications.user_id → users.id
- notifications.job_id → jobs.id
- resume_score_history.user_id → users.id
- saved_jobs.user_id → users.id
- saved_jobs.job_id → jobs.id

10. Backend and API Design

The backend is implemented using FastAPI and SQLAlchemy. Database connectivity is configured through an environment variable named `DATABASE_URL`. The application is served in production by Uvicorn on Railway.

10.1 Representative API Areas

- /login and registration-related endpoints
- /jobs for retrieving and managing job data
- /saved-jobs for saved-job operations
- Application-related endpoints
- Resume/ATS-related endpoints
- Recruiter job-management endpoints

10.2 Database Configuration

The backend reads `DATABASE_URL` from the environment and creates a SQLAlchemy engine. This avoids hard-coding the production database connection in source code and supports deployment-specific configuration.

11. Frontend Design

The frontend is organized as static HTML pages supported by CSS and JavaScript modules. The project includes dedicated pages for the landing experience, authentication, dashboard, jobs, companies, saved jobs, profile, matching, recruiter functionality, candidates, applications and tracking.

11.1 Jobs Page Routing

The production frontend uses Vercel clean URLs. The legacy application path **/jobs-page** is redirected to **/jobs**, which serves the Jobs page. This keeps existing navigation links functional while using a clean production URL.

11.2 Frontend-to-API Configuration

The production frontend is configured to call the Railway backend rather than a localhost API. This is required for the deployed application to communicate with the production backend.

12. Deployment

12.1 Frontend — Vercel

The Vercel project uses the **frontend** directory as its Root Directory and the Other framework preset for the static HTML/CSS/JavaScript application.

12.2 Backend — Railway

The FastAPI backend is deployed as the SwipeX-Backend service on Railway. The public production API domain is:

`https://swipex-backend-production.up.railway.app`

12.3 Database — Neon

The production PostgreSQL database is hosted on Neon. The backend connects to it using the `DATABASE_URL` environment variable.

12.4 Production Frontend

`https://swipex-ai.vercel.app`

13. Testing and Validation

Deployment and data validation were performed during the final project setup.

Test / Validation	Result
Backend production deployment	Successful
Production registration	Successful
Production login	Successful
Jobs HTML availability	Successful
Clean Jobs route /jobs	Successful
/jobs-page compatibility redirect	Successful
Neon database migration	Successful
Foreign-key integrity checks	0 broken references

13.1 Production Data Validation

After migration, the Neon database contained 22 users, 19 jobs, 13 applications, 300 notifications, 12 resume-score-history records and 4 saved-job records. Foreign-key integrity checks returned zero broken references for the defined relationships.

14. Project Data Migration

The project initially used a local PostgreSQL database. A complete local database backup and SQL data export were created before migration. A migration script was prepared to preserve existing records while avoiding collisions with users already present in Neon.

14.1 Migration Strategy

- Back up the local database before making changes.
- Preserve existing Neon users.
- Insert local users with new IDs and create an old-to-new ID mapping.
- Insert local jobs with new IDs and create an old-to-new job mapping.
- Translate foreign keys in applications, notifications, resume history and saved jobs.
- Fix PostgreSQL sequences after insertion.
- Run integrity checks after migration.

The migration completed successfully and the resulting Neon dataset was verified for referential integrity.

15. Future Enhancements

- Improve recommendation quality with more advanced ranking and personalization models.
- Add richer resume parsing and skill extraction.
- Introduce stronger recruiter analytics and candidate ranking.
- Add automated email/push notifications for application events.
- Improve accessibility and responsive behavior across devices.
- Add automated CI/CD checks for frontend and backend changes.
- Expand observability with structured logging and application monitoring.
- Introduce more advanced search, filtering and recommendation explanations.

16. Conclusion

SwipeX demonstrates an end-to-end web application architecture for intelligent job discovery and career assistance. The completed system connects a browser-based frontend with a FastAPI backend and PostgreSQL database, while supporting both job-seeker and recruiter workflows.

The production deployment separates the frontend, backend and database into appropriate hosting services. The local project data was migrated to Neon with relationship integrity preserved, and the production Jobs route and authentication workflow were validated.

17. Repository and Live Deployment

GitHub Repository

Official repository: [springboardmentor27200x-wq / -Swipe-Based-Intelligent-Job-Discovery-and-Career-Assistance-Platform](#)

Active development branch

`Nidrashri-V`

Latest deployment-related commit

`632ed9c – Fix Vercel frontend routing`

Live Frontend

<https://swipex-ai.vercel.app>

Live Backend

<https://swipex-backend-production.up.railway.app>

Note: Credentials, database passwords, API secrets and other sensitive environment values are intentionally excluded from this report and should not be committed to the repository.